Procédures et Fonctions Stockées en MySQL

I. Les Procédures Stockées

Définition

Une **procédure stockée** est un **programme enregistré dans la base de données**. Elle permet d'exécuter un ensemble d'instructions SQL en une seule commande, ce qui facilite la réutilisation et l'automatisation des tâches.

Structure générale

```
CREATE PROCEDURE nom_procedure (liste_des_paramètres)
BEGIN
   -- Déclarations
   DECLARE nom_var TYPE;

   -- Affectations
   SET nom_var = valeur;
   -- ou
   SELECT ... INTO nom_var;

   -- Instructions SQL
   ...
END;
```

Appel de la procédure :

```
CALL nom_procedure(valeurs_des_paramètres);
```

Suppression :

```
DROP PROCEDURE nom_procedure;
```

Exemple simple

Afficher tous les stagiaires :

```
DELIMITER /
CREATE PROCEDURE pc1()
BEGIN
    SELECT * FROM stagiaires;
END /
DELIMITER ;

CALL pc1();
```

II. Les Fonctions Stockées

Définition

Une **fonction stockée** est similaire à une procédure, mais elle **retourne une valeur** (comme une fonction en mathématiques).

Structure générale

```
CREATE FUNCTION nom_fonction (liste_des_paramètres)
RETURNS type_de_retour
BEGIN
    DECLARE variable TYPE;
    SET variable = valeur;
    RETURN variable;
END;
```

Appel de la fonction :

```
SELECT nom_fonction(valeurs_des_paramètres);
```

Suppression :

```
DROP FUNCTION nom_fonction;
```

Exemple simple

```
DELIMITER /
CREATE FUNCTION somme(a INT, b INT)
RETURNS INT
BEGIN
    DECLARE s INT;
    SET s = a + b;
    RETURN s;
END /
DELIMITER ;

SELECT somme(1, 2);
```

III. Étude de Cas : Gestion des Stagiaires

1. Création de la table

```
CREATE TABLE stagiaires (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    ville VARCHAR(100)
);
```

---

2. Insertion de données

```
INSERT INTO stagiaires (nom, ville) VALUES
('Ali', 'Casablanca'),
('Sara', 'Rabat'),
('Khalid', 'Casablanca'),
('Mouna', 'Marrakech'),
('Youssef', 'Rabat');
```

---

3. Création des procédures

➢ a. Afficher les stagiaires d'une ville donnée

```
DELIMITER /
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_param
VARCHAR(100))
BEGIN
    SELECT * FROM stagiaires WHERE ville = ville_param;
END /
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
```

➢ b. Compter le nombre total de stagiaires

```
DELIMITER /
CREATE PROCEDURE compter_stagiaires()
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total FROM stagiaires;
    SELECT total AS 'Nombre de stagiaires';
END /
DELIMITER ;

CALL compter_stagiaires();
```

---

4. Création d'une fonction

➢ a. Nombre de stagiaires par ville

```
DELIMITER /
CREATE FUNCTION nombre_stagiaires_par_ville(ville_param VARCHAR(100))
RETURNS INT
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total FROM stagiaires WHERE ville =
ville_param;
    RETURN total;
END /
DELIMITER ;

SELECT nombre_stagiaires_par_ville('Rabat');
```

Ces deux commandes permettent d'afficher la liste des **procédures stockées** et des **fonctions stockées** dans la base de données MySQL actuellement sélectionnée.

```
SHOW PROCEDURE STATUS WHERE Db = DATABASE();

SHOW FUNCTION STATUS WHERE Db = DATABASE();
```

5. Suppression des procédures et fonctions

```
DROP PROCEDURE IF EXISTS afficher_stagiaires_par_ville;
DROP PROCEDURE IF EXISTS compter_stagiaires;
DROP FUNCTION IF EXISTS nombre_stagiaires_par_ville;
```

IV. Les Structures de Contrôle et Boucles

IF

Permet d'exécuter des instructions selon une condition :

```
IF condition THEN
    instructions;
ELSE
    autres_instructions;
END IF;
```

CASE

Alternative à IF pour plusieurs conditions :

```
CASE variable
    WHEN valeur1 THEN instruction1;
    WHEN valeur2 THEN instruction2;
    ELSE instruction_par_defaut;
END CASE;
```

REPEAT

Boucle exécutée **au moins une fois**, puis répétée jusqu'à ce qu'une condition devienne vraie.

```
REPEAT
    instructions;
UNTIL condition
END REPEAT;
```

WHILE

Boucle exécutée **tant qu'une condition est vraie**.

```
WHILE condition DO
    instructions;
END WHILE;
```

V. Exemple Pratique : Facturation de l'eau

Objectif

Créer un programme qui :

1. Insère 100 factures avec des consommations croissantes.
2. Calcule automatiquement le montant à payer selon des tranches.

Barème :

- 0 – 10 m³ → 1 € par m³
- 11 – 30 m³ → 1.5 € par m³
- 31 m³ et + → 2 € par m³

Création de la Table

```
CREATE TABLE facture (
    id_facture INT AUTO_INCREMENT PRIMARY KEY,
    nom_client VARCHAR(255),
    qte_eau INT
);
```

1-Créer un programme qui insère 100 factures avec des quantités d'eau croissantes (incrémentées de 1).

```
DELIMITER /

CREATE PROCEDURE inserer_factures()
BEGIN
    DECLARE compteur INT DEFAULT 1;
    WHILE compteur <= 100 DO
        INSERT INTO facture (nom_client, qte_eau)
        VALUES (CONCAT('Client', compteur), compteur);
        SET compteur = compteur + 1;
    END WHILE;
END /

DELIMITER ;
```

Exécution : CALL inserer_factures()/

Exemple de fonction

```
DELIMITER /
CREATE FUNCTION calculer_montant(qte INT)
RETURNS DECIMAL(10,2)
BEGIN
    DECLARE montant DECIMAL(10,2);

    IF qte <= 10 THEN
        SET montant = qte * 1;
    ELSEIF qte <= 30 THEN
        SET montant = (10 * 1) + (qte - 10) * 1.5;
    ELSE
        SET montant = (10 * 1) + (20 * 1.5) + (qte - 30) * 2;
    END IF;

    RETURN montant;
END /
DELIMITER ;

SELECT id_facture, nom_client, qte_eau,
       calculer_montant(qte_eau) AS montant_a_payer
FROM facture;
```

VI. Les Paramètres IN, OUT et INOUT (⚙ notion avancée)

1. Paramètre IN

- Sert à **transmettre une valeur à la procédure**.
- C'est la **valeur d'entrée** (par défaut dans MySQL).
  Exemple :

```
CREATE PROCEDURE supprimer_stagiaire(IN p_id INT)
BEGIN
    DELETE FROM stagiaires WHERE id = p_id;
END;
```

---

2. Paramètre OUT

- Sert à **récupérer une valeur calculée** par la procédure.
  Exemple :

```
CREATE PROCEDURE compter_stagiaires_out(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM stagiaires;
END;

CALL compter_stagiaires_out(@nb);
SELECT @nb AS total_stagiaires;
```

---

3. Paramètre INOUT

- Sert à **transmettre une valeur**, la **modifier**, puis la **renvoyer**.
  Exemple :

```
CREATE PROCEDURE augmenter_stock(INOUT qte INT, IN p_id INT)
BEGIN
    UPDATE produits SET stock = stock + qte WHERE id = p_id;
    SELECT stock INTO qte FROM produits WHERE id = p_id;
END;

SET @ajout = 10;
CALL augmenter_stock(@ajout, 1);
SELECT @ajout AS nouveau_stock;
```

Résumé visuel

Type	Direction	Rôle principal	Exemple
IN	Entrée	Envoi d'une valeur	ID à supprimer
OUT	Sortie	Retour d'une valeur calculée	Nombre total
INOUT	Entrée/Sortie	Envoi puis modification	Mise à jour d'un stock